

Complete Server Architecture, App-by-App Functionality & Simulation Audit Manual

This exhaustive technical and conceptual reference manual documents every facet of the backend server. Structured into distinct, self-contained chapters for every Django application, this document provides parallel perspectives: high-level plain-English workflows and real-world analogies for non-technical readers, alongside deep mathematical models, database schemas, REST API contracts, cryptographic protocols, and state machines for software architects and systems engineers. Chapter 10 presents an unfiltered audit of all production vs simulated subsystems.

Backend Framework	Django 5.x, Django REST Framework (DRF), SimpleJWT
Database Layer	PostgreSQL with atomic transactions and row-level locking (select_for_update)
Networking & Tunneling	Dynamic Relay Port Pooling (40000-50000), Reverse SSH Proxy handshakes
Cryptographic Security	RSA 2048-bit and Ed25519 OpenSSH keypair generation via cryptography.hazmat
Billing Engine	Atomic Escrow Hold & Settle, 90/10 Host-Platform Split, Fractional-Second Decimal Math
API Documentation	OpenAPI 3.0, Swagger UI, Redoc, Django Administration Portal

Executive Overview: The Decentralized GPU Cloud

■ Non-Technical Concept (In Plain English): The Peer-to-Peer GPU Paradigm

Large-scale Artificial Intelligence training, Large Language Models (LLMs), and 3D computer graphics require massive GPU power. Centralized hyperscalers (AWS, Azure, GCP) charge premium rates that are inaccessible to independent developers, students, and small enterprises. Conversely, millions of high-performance gaming rigs, workstations, and private server clusters sit idle for over 18 hours every day.

This platform establishes a **decentralized computing marketplace**: Individuals with GPUs (**Hosts**) monetize their hardware by sharing compute cycles. Consumers needing compute (**Renters**) search, filter, and rent GPUs with zero long-term commitments. The central server ensures safety: it locks funds in escrow, verifies machine uptime, allocates encrypted networking tunnels, streams live hardware diagnostics, and guarantees instant payouts upon job completion.

■■ Technical Architecture & Under-the-Hood Mechanics: Central Server Responsibilities & Architectural Planes

The central Django backend acts as the authoritative coordinator without ever storing or intercepting user payload data:

- **Control Plane:** Handles user authentication, RBAC authorization, marketplace listings, hardware capability discovery, and session state machines.
- **Financial & Settlement Plane:** Pre-authorizes escrow holds, computes precise usage costs down to fractional seconds, deducts 10% platform fees, credits 90% net earnings to host wallets, and issues instantaneous refunds for unused booked time.
- **Connectivity & Cryptographic Plane:** Manages dynamic relay port leases, generates ephemeral RSA/Ed25519 keypairs, coordinates reverse SSH handshakes, and issues one-line connection strings.
- **Telemetry & Governance Plane:** Ingests periodic hardware health metrics (temperature, VRAM, GPU compute utilization), penalizes unreliable nodes, and provides moderation safeguards.

Chapter 1: System Architecture, Core Workflows & Networking Protocols

The central engineering challenge in a peer-to-peer compute platform is **NAT Traversal and Firewall Bypass**. Host machines reside behind residential routers, dynamic IP addresses, and carrier-grade NATs (CGNAT), making incoming connections impossible without manual port forwarding. The platform resolves this via a centralized **SSH Reverse-Relay Architecture**.

End-to-End Operational Lifecycle

Lifecycle Step	Non-Technical Perspective	Technical Engine Execution & Database Flow
1. Discovery	Renter filters the marketplace by GPU model, VRAM size, hourly rate, or host reliability rating.	GET /api/gpus/ queries the GPU model with active filters (is_available=True, host_status="online").
2. Reservation & Hold	Renter books 5 hours. The total estimated cost is locked in escrow in the renter's wallet.	POST /api/sessions/create/ validates wallet available_balance. Invokes BillingService.hold_funds() locking hold_amount. Calls RelayService.allocate_port() with row-level lock (select_for_update) on RelayPort table.
3. Host Polling	The host background worker daemon detects a new rental job and accepts it.	Host daemon calls GET /api/sessions/host/pending/. Retrieves session_id, leased relay_server_port, and ephemeral RSA private key (relay_auth_key).
4. Container & Tunnel Setup	Host node spins up an isolated environment and establishes an encrypted connection tunnel.	Host daemon initializes isolated Docker container with GPU passthrough (--gpus all), opens reverse SSH tunnel (ssh -R :localhost:22 renter@), and sends status ACTIVE via PATCH /api/sessions/host/status/.
5. Active Session & Renter Access	The rental officially goes live. The renter pastes the connection command into their terminal to start working.	Server stamps active_time = timezone.now(), issues ssh_connection_string = "ssh renter@ -p ", and notifies renter. The billing clock is now running.
6. Telemetry Monitoring	Live hardware performance stats (temperature, VRAM usage, utilization %) stream to both dashboards.	Host daemon queries local nvidia-smi and transmits metrics to POST /api/sessions/host/heartbeat/, inserting real-time SessionMetric database rows.
7. Session Stop & Settlement	Renter stops the session early. Unspent funds are refunded immediately, and the host receives 90% payment.	POST /api/sessions/stop/ triggers BillingService.process_rental_payment(): computes exact elapsed duration down to seconds, releases the escrow hold, deducts actual cost, credits 90% to host wallet, records 10% platform fee, refunds unused balance, marks GPU available, and releases relay port.

Chapter 2: The `users` App — Identity, RBAC, Profiles & Node Security

■ Non-Technical Concept (In Plain English): User Identities & Dedicated Node Security

Users can register as Renters (who buy compute), Hosts (who sell compute), or Both. Because host machines need to run automated worker daemons in the background 24/7, the system issues machine-specific **Host API Keys** (host...). This prevents the host from ever needing to store their personal email password on an unattended machine.

Technical Models, Fields & Logic

- **User Model:** Custom user model extending AbstractBaseUser and PermissionsMixin. Key fields: id (UUID PK), email (unique, indexed), role (renter, host, both, admin), host_api_key (unique token for daemon auth), is_email_verified, is_active.
- **HostProfile Model:** One-to-one extension profile created automatically for hosts. Attributes include: gpu_name, vram_gb, driver_version, cuda_version, status (online, offline, restricted, suspended), uptime_percentage, reliability_score (starts at 100), penalty_points, total_earnings, pending_payout, total_rental_hours, auto_accept (boolean), max_rental_hours.
- **HostProfile.apply_penalty(points, reason):** Deducts reliability score (score = max(0, 100 - penalty_points * 2)), updates status to restricted if points ≥ 50 or suspended if points ≥ 100, and records a HostPenaltyLog entry.
- **LoginAttempt Model:** Enforces brute-force defense by recording IP-level attempt counts and locking offending IPs upon multiple consecutive failures.
- **PasswordResetToken & EmailVerificationToken:** Provide cryptographically secure, time-expiring UUID tokens for identity recovery.

Endpoint	Auth	Functional Action Description
POST /api/auth/register/	Public	Registers user, hashes password via Argon2/PBKDF2, provisions Wallet, issues email token.
POST /api/auth/login/	Public	Verifies credentials, checks LoginAttempt lockout status, returns JWT access/refresh tokens.
POST /api/auth/token/refresh/	Public	Rotates expired JWT access token using valid refresh token.
GET/PATCH /api/auth/profile/	JWT Auth	Fetches or modifies authenticated user personal and contact details.
GET/PATCH /api/auth/host/profile/	Host Auth	Retrieves or configures host hardware profile, auto-accept mode, and schedule.
POST /api/auth/host/api-key/	Host Auth	Generates a secure, random host_ daemon token.
POST /api/auth/host/api-key/validate/	Public	Host daemon validates its API key on boot before polling pending rental queues.
POST /api/auth/password-reset/request/	Public	Issues secure password reset token with 24-hour expiration.
POST /api/auth/password-reset/confirm/	Public	Validates reset token and sets new hashed password.

Chapter 3: The `gpust` App — Fleet Inventory, Specs & Heartbeat Monitoring

■ Non-Technical Concept (In Plain English): GPU Marketplace & Zero-Stale-Listing Protection

Hosts register the exact hardware specs of their graphics cards (e.g. NVIDIA RTX 4090, 24GB VRAM, CUDA 12.2) and define their own hourly rental price in NPR. The platform guarantees that renters never attempt to book an offline card: if a host machine powers off or disconnects, the system detects the missing heartbeat and delists the card from the marketplace within minutes.

Data Models, Availability Rules & Heartbeat Tracking

- **GPU Model:** Primary inventory entity. Key attributes: host (FK to HostProfile), gpu_name (e.g., RTX 4090), vram_gb (integer VRAM in GB), cuda_cores, memory_bandwidth, compute_capability, driver_version, cuda_version, price_per_hour (validated $\geq$ NPR 0.01), is_available (boolean), current_session_id (UUID), location, total_rental_hours, total_earnings, total_sessions.
- **The is_rentable Property:** Evaluates dynamic rentability:
return (self.is_available and self.current_session_id is None and self.host.status == "online")
- **GPU Lifecycle Methods:**
 - mark_rented(session_id): Sets is_available=False and records current_session_id.
 - mark_available(): Clears current_session_id and sets is_available=True.
 - update_stats(hours, earnings): Accumulates total rental hours and revenue in Decimal precision.
- **Heartbeat Monitor (GPUHeartbeatView):** Host background agents submit periodic telemetry to POST /api/gpus/heartbeat/. The server updates host.last_heartbeat = now() and marks host.status = "online".

Chapter 4: The `sessions` App — State Engine, SSH Relay & Telemetry

■ Non-Technical Concept (In Plain English): The Core Heartbeat of the Entire Platform

A Session is the legal and technical rental contract. It begins the instant a user selects a GPU and duration. It supervises container creation on the host, establishes the encrypted SSH communication bridge, records live temperature and GPU utilization charts, and calculates the exact final bill when the session stops.

Complete 9-Stage Session State Machine

State Code	System Meaning & Daemon Trigger	Financial / Billing Clock Status
pending	Renter creates request; waiting for host pickup.	Funds placed on HOLD (reserved, not deducted).
starting	Host daemon polls session and begins container prep.	Billing paused (clock not running).
container_running	Docker container with GPU isolation is running on host.	Billing paused.
tunnel_connecting	Host establishes reverse SSH tunnel to relay port.	Billing paused.
active	Tunnel confirmed live. SSH credentials issued to renter.	BILLING CLOCK STARTS (active_time = now()).
stopping	Stop initiated by renter or duration expired.	Usage computed down to fractional seconds.
completed	Container destroyed; relay port released.	Settled: 90% to host, 10% platform, refund unused.
failed	Host error or setup failure during startup.	100% Full Refund: Hold released, zero fee.
terminated	Terminated early by admin or host offline dropout.	Refund processed; compensation applied if insured.

Cryptographic Key Generation & Port Management Services

- **RelayService.allocate_port(session_id):** Finds the lowest available port in range 40000-50000 using `RelayPort.objects.select_for_update().filter(status="free").first()`, leasing the port atomically to guarantee zero duplicate port collisions under high concurrent load.
- **RelayService.generate_ssh_keypair():** Uses Python's `cryptography.hazmat.primitives.asymmetric.rsa` library to generate an authentic 2048-bit RSA keypair with 65537 public exponent. Formats private key in PEM format and public key in OpenSSH wire format.
- **SessionMetric Ingestion:** Hosts report telemetry via `POST /api/sessions/host//heartbeat/`. Captures: `gpu_temperature_c`, `gpu_utilization_pct`, `memory_used_mib`, `power_usage_w`, `fan_speed_pct`, `cpu_usage_pct`, `ram_usage_mb`.

Chapter 5: The `wallets` App & Financial Engine

■ Non-Technical Concept (In Plain English): How Money Moves: The Two-Stage Hold & Settle Model

When a user rents a GPU for 5 hours at NPR 100/hr, NPR 500 is placed on **HOLD** (locked in escrow). The money is NOT yet deducted. If the user stops after 1.5 hours, the system charges NPR 150, instantly returns NPR 350 back to their available balance, and transfers NPR 135 (90%) to the host.

Mathematical Financial Rules & Transaction Ledger

All financial operations execute inside atomic database transactions (@transaction.atomic) with select_for_update() row locks on the Wallet table:

Financial Operation	Mathematical Formula & Decimal Logic	Transaction Ledger Type
1. Pre-Authorization	wallet.hold_amount += total_amount available_balance = balance - hold_amount	type = "hold"
2. Usage Calculation	used_hours = (end_time - active_time).total_seconds() / 3600 actual_cost = used_hours * price_per_hour	Computed in Decimal
3. Release Hold	wallet.hold_amount -= total_amount	Hold cleared
4. Unspent Refund	refund = total_amount - actual_cost if refund > 0: wallet.balance += refund	type = "refund"
5. Host Payout Split	platform_fee = actual_cost * Decimal("0.10") (10%) host_earnings = actual_cost - platform_fee (90% to host)	type = "rental_payment" type = "host_earning" type = "platform_fee"
6. Session Failure Release	wallet.hold_amount -= total_amount (0 deduction)	type = "release_hold"

Chapters 6 to 9: Notifications, Reviews, Dashboard & Governance

Chapter 6: The `notifications` App — Real-Time Alerting Engine

The notifications app delivers transactional, operational, and security alerts to renters and hosts. The centralized NotificationService provides standardized asynchronous dispatchers:

- `notify_session_started(session)`: Transmits SSH connection string and confirmation to renter.
- `notify_session_completed(session, billing)`: Issues itemized billing receipt and payout notice.
- `notify_session_terminated(session, reason)`: Warns of node dropouts or administrative interventions.
- `notify_refund_processed(user, amount)`: Confirms balance re-crediting after session stops.
- `notify_penalty_applied(host, points, reason)`: Alerts hosts of reliability score deductions.

Chapter 7: The `reviews` App — Verified Multi-Criteria Reputation

The reviews app safeguards marketplace trust through verified feedback tied directly to completed sessions:

- **Multi-Criteria Ratings (1 to 5 Stars)**: Overall Rating, Host Communication Rating, Reliability Rating, and GPU Hardware Performance Rating.
- **Verification Guard**: A review can strictly only be authored for a completed session by the authenticated renter.
- **ReviewQuerySet.summary_stats()**: Custom ORM manager computing average ratings, verified counts, and 1-5 star distributions in a single SQL aggregate query.
- **Host Response Mechanism**: Hosts can submit public responses to reviews via HostReviewResponseView.

Chapter 8: The `dashboard` App — Role-Aware Analytics & Security Logs

The dashboard app powers the user interface with tailored data streams:

- **Renter Dashboard**: Active workloads, available wallet balance, recent sessions, cumulative compute spend.
- **Host Dashboard**: GPU fleet utilization (total/available/rented), revenue (today, this week, this month), reliability score, uptime %, and per-GPU hourly utilization charts.
- **Admin Analytics**: Gross Merchandise Value (GMV), platform fee collections, active host nodes, active session counts.
- **Activity Auditing (UserActivityLog)**: Immutable audit trail logging user interactions (login, session start, profile edit, payment) with IP and user agent metadata.

Chapter 9: The `admin\_panel` App — Platform Governance & Moderation

Provides platform administrators with oversight and dispute resolution mechanisms:

- **Emergency Session Kill Switch**: Force-terminates rogue or unresponsive sessions and triggers automated refunds.
- **Host Node Moderation**: Inspects node reliability scores, resolves penalty appeals, and suspends abusive hosts.
- **Financial Dispute Handling**: Issues manual wallet compensation and refund adjustments with audit logging.

Chapter 10: Comprehensive Platform Simulation Audit

■ Non-Technical Concept (In Plain English): Full Transparency: Simulated vs Production-Ready Subsystems

To facilitate transparent technical audits, academic presentations, hackathon evaluations, and commercial roadmap planning, this chapter provides an unfiltered matrix detailing which platform components are 100% real and production-functional versus which modules are simulated or mock-backed.

Subsystem / Component	Current Status	Detailed Technical Reality & Simulation Analysis
Cryptographic SSH Keypair Generator	100% REAL	Uses genuine Python cryptography.hazmat primitives to mathematically generate real 2048-bit RSA and Ed25519 OpenSSH keys and PEM blocks.
Relay Port Leasing Engine	100% REAL	Real PostgreSQL database table managing ports 40000-50000 with row-level locking (select_for_update) preventing double allocations under concurrent load.
Financial Ledger & Billing Engine	100% REAL	Fully functional double-entry accounting in PostgreSQL: atomic hold, release, fractional-second usage billing, 90/10 split, and instant refunds.
Marketplace Query & Ratings Engine	100% REAL	Full SQL filtering by VRAM, price, CUDA version, and real-time calculation of host reputation scores and review distributions.
SSH Reverse-Tunnel Relay Server	SIMULATED	The database orchestrates port leases and generates valid SSH strings. The external reverse-tunnel proxy server (Frp / SSH jump host) endpoint defaults to 127.0.0.1 / RELAY_HOST.
Host Container Provisioning Daemon	MOCK-ENABLED	Host polling and status update API endpoints are fully implemented. For testing without physical GPUs, status transitions can be stepped manually via REST calls.
GPU Hardware Telemetry Ingestion	HYBRID	Backend storage and aggregation of temperature, memory, and load are real; values are supplied by host heartbeat payloads (simulated or from nvidia-smi).
Fiat Payment Gateways	SIMULATED	Stripe/eSewa/Khalti balance crediting operates on internal wallet balance. Live external bank webhooks are plug-and-play ready.
Work Protection Insurance	HYBRID	The 20¢/hour insurance fee and policy tracking are active; automated compensation claims are administrator-reviewed.

Roadmap to 100% Bare-Metal Production Deployment

Transitioning from the current demonstration architecture to commercial nationwide production requires exactly three operational steps:

- 1. Deploy a Dedicated Public Relay Server:** Configure a Linux cloud instance with OpenSSH GatewayPorts yes or frps (Fast Reverse Proxy server) and point Django's RELAY_HOST setting to its public IP address.
- 2. Run the Python Host Agent Daemon on Physical Machines:** Execute the host worker script on nodes equipped with NVIDIA drivers and Docker (nvidia-container-toolkit). The agent will automatically call nvidia-smi and establish live reverse SSH tunnels.
- 3. Attach Live Payment Gateway Credentials:** Insert live Stripe / eSewa API keys into Django environment variables to enable automated credit card and bank balance top-ups.